



VR Builder

Open source toolkit for VR creation

Table of Contents

Introduction	
Requirements	
Installation	
Updating	
Quick Start	
Process Scene Objects	
XR Rig	
Controller visualization	
Locomotion	
Tunneling Vignette	
Behaviors	
Animation	
Move Object	
Scale Objects	
Environment	
Disable Objects	
Enable Objects	
Disable Components	
Enable Components	
Start Particle Emission	
Stop Particle Emission	
Unsnap Object	
Guidance	
Play Audio File	
Play Text-to-Speech Audio	
Highlight Objects	
Spawn Confetti	
Utility	
Behavior Sequence	
Delay	
Set Parent	
Load Scene	
Conditions	
Environment	
Objects in Collider	

[Object Nearby](#)

[Interaction](#)

[Grab Object](#)

[Release Object](#)

[Snap Object](#)

[Touch Object](#)

[Poke Object](#)

[Use Object](#)

[Locomotion](#)

[Teleport](#)

[Utility](#)

[Timeout](#)

[Online Tutorials](#)

[Acknowledgements](#)

[Contact and Support](#)

Introduction

VR Builder helps you create interactive VR applications better and faster. By setting up a Unity scene for VR Builder, you will pair it with a VR Builder *process*. Through the VR Builder process, you can define a sequence of actions the user can take in the scene and the resulting consequences.

You can easily edit a process without coding through VR Builder's process editor. The process editor is a node editor where the user can arrange and connect the *steps* of the process. Each step is a different node and can include any number of *behaviors*, which make things happen in the scene. Likewise, a step will have at least one *transition* leading to another step. Every transition can list several *conditions* which have to be completed for the transition to trigger. For example, step B can be reached only after the user has grabbed the object specified in step A.

Behaviors and conditions are the "building blocks" of VR Builder. Several of them are provided in the free open-source version already. Additional behaviors and conditions are available in the commercial version, available on the [Unity Asset Store](#) and to Enterprise customers and Content Partners. Since VR Builder is open source and has an open API, you can always write your own behaviors and conditions as well.

Behaviors and conditions can interact only with *process scene objects*. These are game objects in the scene which have a `Process Scene Object` component on them.

The interaction capabilities of a process scene object can be increased by adding *scene object properties* to it. For example, adding a `Grabbable Property` component to a game object will let VR Builder know that the object is grabbable, and when it is grabbed.

Normally it is not necessary to add properties manually to an object. When an object is dragged in the inspector of a condition or behavior, the user has the option to automatically configure it with a single click.

Where possible, properties try to add and configure required components by themselves. If you add a `Grabbable Property` to a game object, this will automatically be made grabbable in VR (it still needs to have a collider and a mesh, of course).

This makes it very easy to start from some generic assets and build a fully interactive scene.

Requirements

VR Builder supports Unity 6 or later. It uses the Universal Render Pipeline (URP) by default. Using the Built-In Render Pipeline is still possible, but you will need to replace invalid material shaders, such as those on controller models in the Rig.

If you are using an older Unity version, we recommend to fall back to VR Builder 4.x which officially supports versions prior to Unity 6.

VR Builder works out of the box with any headset compatible with Unity's XR Interaction Toolkit 3.x.

Installation

You can get VR Builder from the [Unity Asset Store](#) or from our [GitHub](#). In the first case, you will download and import the VR Builder package from the Package Manager. From GitHub, you can either download a released .UnityPackage or pull the repository and import the `package.json` file in the Package Manager.

If you are creating a new project, select `Universal 3D` template. Note that it is not necessary to choose the `VR` template as VR Builder will configure the project for VR automatically.

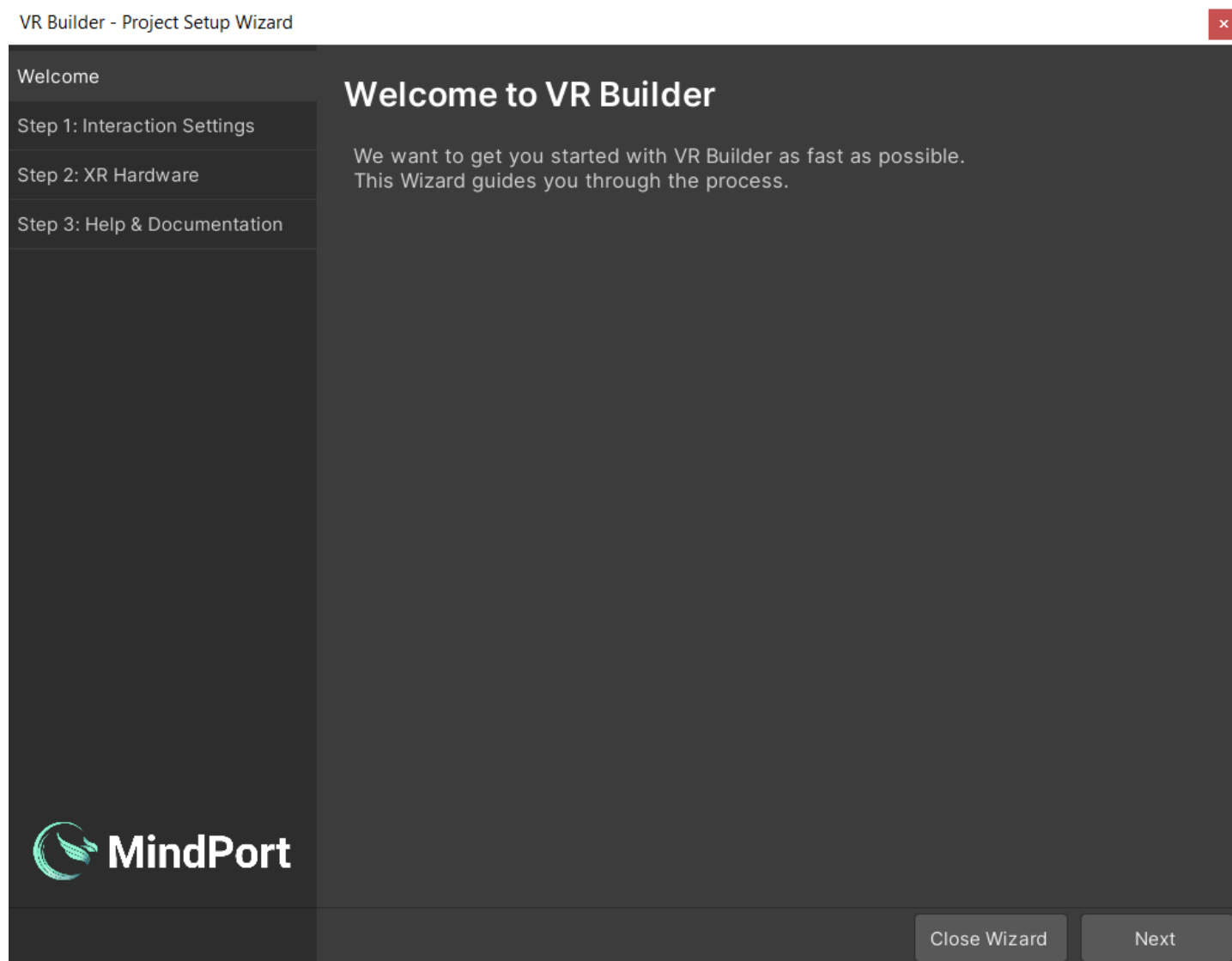
After importing, VR Builder will compile and import some dependencies, then the Project Setup Wizard will appear.

Guide Through the Project Setup Wizard

The Setup Wizard helps to get new or existing Unity scenes set up and ready to be used with VR Builder.

Welcome

In the following we will guide you through the different tabs of the VR Builder Setup Wizard. Click `Next` to proceed to the interaction settings page setup page.



Interaction Settings

Here you can configure some default settings related to VR Builder interactions. Right now, the only available setting will determine whether a newly created grabbable object will use physics or not. You can also change these options at any time in `Project Settings > VR Builder > Settings`.

Welcome

Step 1: Interaction Settings

Step 2: XR Hardware

Step 3: Help & Documentation

Interaction Settings

You can choose how VR Builder configures objects that it makes grabbable. If you check the option below, the object will ignore physics and stay in place when the user releases it, even in mid-air. Otherwise, it will fall to the floor due to gravity.

☐ Make newly created grabbables ignore physics

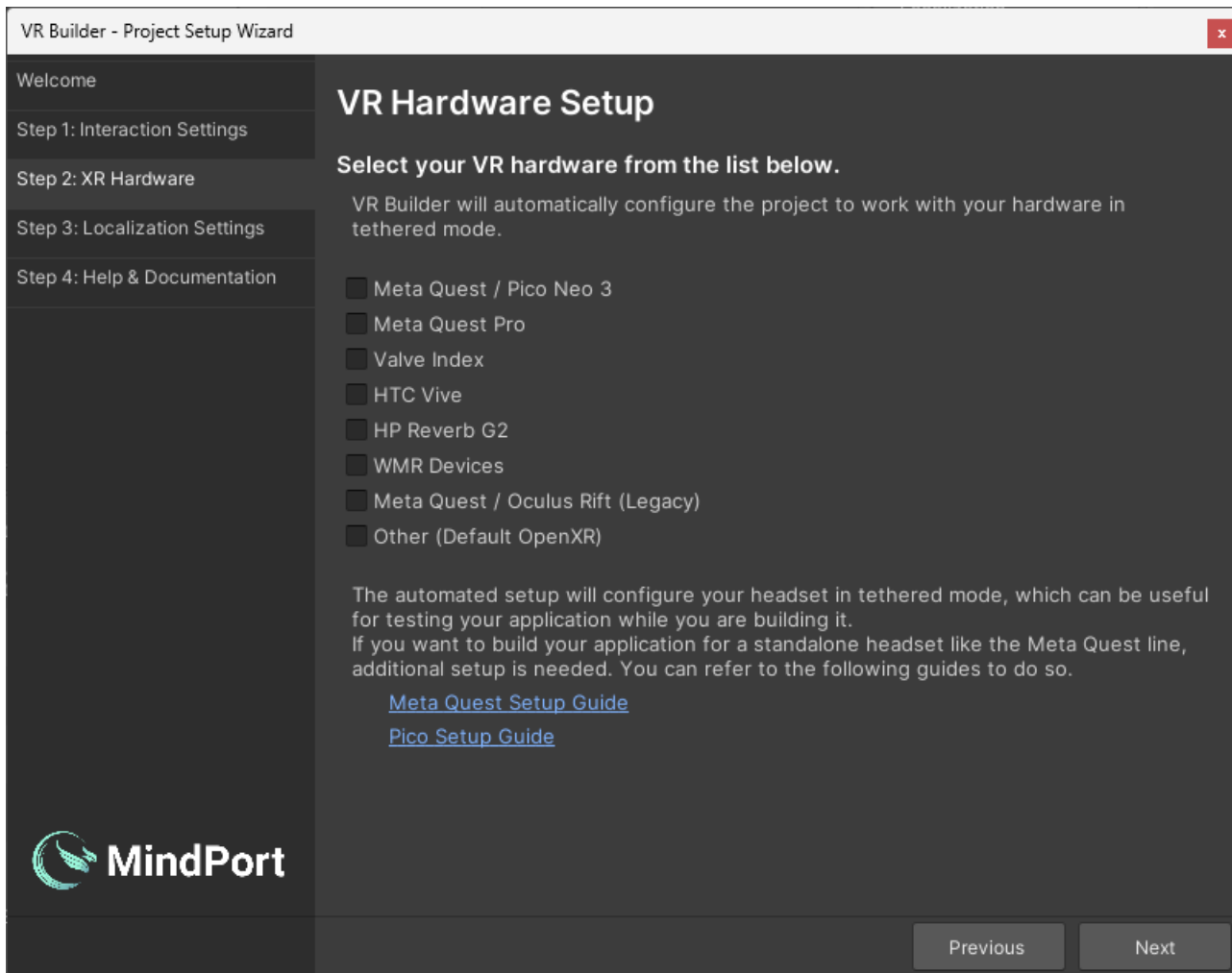
 **MindPort**

Previous

Next

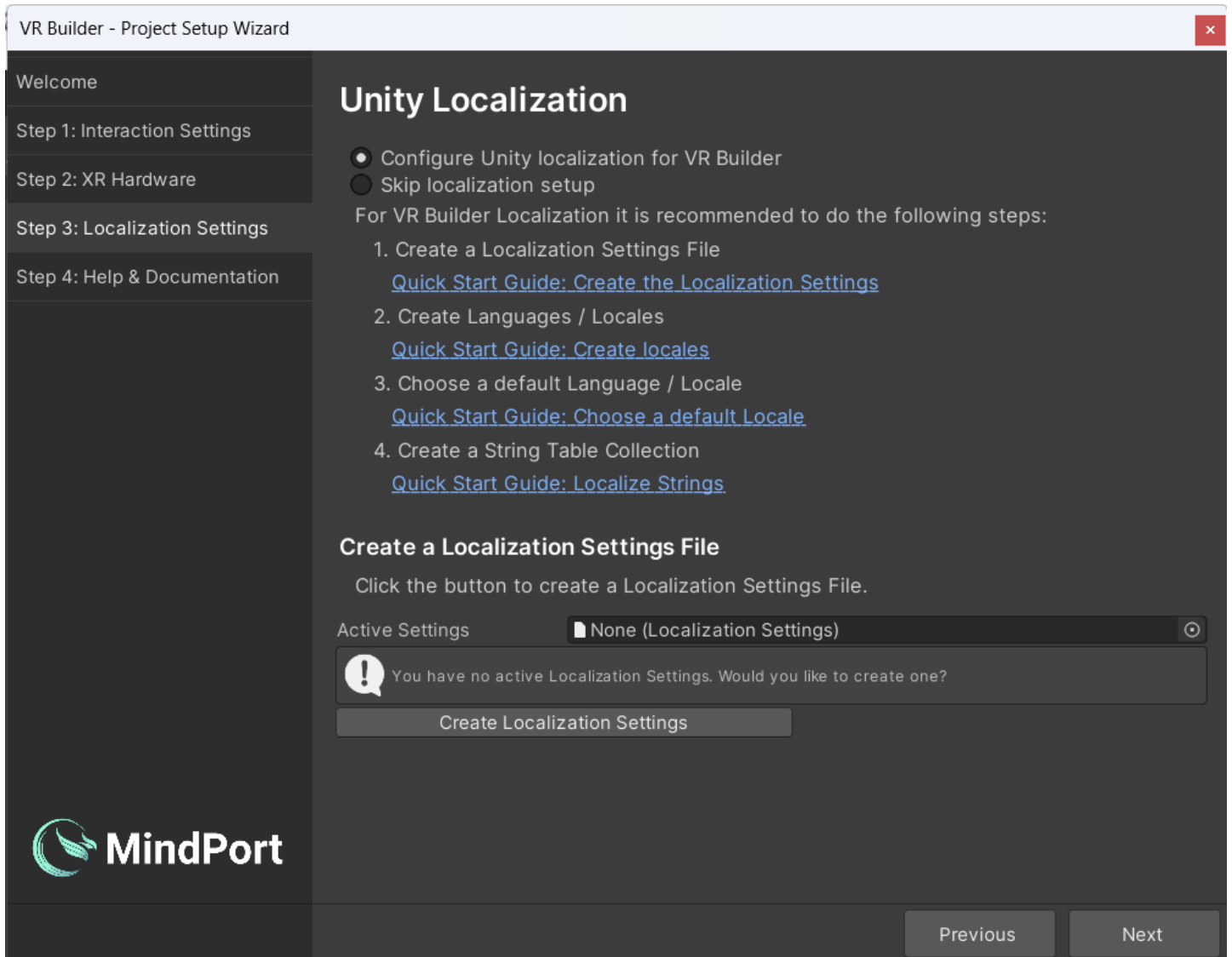
XR Hardware Setup

Then, if it's not configured already, you will be able to configure your project to work with your VR hardware. If your hardware is not listed but supports OpenXR, select Other. You might need to finish the configuration under Project Settings -> XR Plug-in Management. If your device is not supported, please reach out to us in the VR Builder Community. The packages for the selected hardware will be imported after closing the wizard.



Unity Localization

Finally you must choose whether to set up localization support or skip it for now. VR Builder supports localization through the Unity Localization package. When setting up localization, the wizard will guide you through the manual steps required for a localized project. It provides a useful checklist and some shortcuts to make the task easier. It also has links that redirect to the relevant Unity documentation. If you decide to skip this setup, VR Builder will work in a single language. You can perform localization setup at any time. We encourage you to use Unity Localization, even if you use just one language. With Unity Localization, you will have all your texts in one place inside the Unity Localization Tables, which in turn will give you the possibility to export and refine them and import them back into Unity. Note that as of January 2024, we do not support asset localization tables. See [Resources path/Key](#) for Play Audio Files in the Documentation on how to localize audio files.



Help & Documentation

This page lists some useful resources to get you started with VR Builder. There is also a checkbox which lets you choose if the demo scene should be loaded after closing the wizard. After clicking Finish, VR Builder is set up! You will need to wait a few seconds for packages to import if you selected a headset from the hardware list. Then, feel free to try out the demo scene or create your own scene by selecting `Tools > VR Builder > Scene Setup Wizard...`.

If the demo scene or any other asset looks solid magenta there is an issue with the shader of the material. The Demo Scenes in VR Builder use the Universal Render Pipeline (URP). If the project uses a different render pipeline, they will not display properly.

VR Builder - Project Setup Wizard

Welcome

Step 1: Interaction Settings

Step 2: XR Hardware

Step 3: Localization Settings

Step 4: Help & Documentation

Help & Documentation

Have a look at the following resources for further information.

Help

[Documentation](#)[Tutorials](#)[Roadmap](#)

Community

[Community](#)[Contact us](#)[GitHub](#)

Review

If you like what we are doing, you can help us greatly by leaving a positive review on the Unity Asset Store!

[Leave a review](#)

Demo Scene

☒ Load the demo scene after closing the wizard.

!

VR Builder will automatically copy the process JSON to the StreamingAssets folder before opening the demo scene.

You can access the menu under Tools > VR Builder to load a demo scene at any time or create a new VR Builder scene.

 **MindPort**

Previous

Finish

10 of 57

Updating

To update to a new version of VR Builder, it is recommended to remove the package before importing the new one. It is also good practice to backup/commit your project before updating the package.

Removing the old package is necessary in order to ensure that obsolete files are removed. When an Asset Store package is updated, it only adds new files and updates the existing ones - no files are removed, which could result in redundant or conflicting code.

To remove the asset, open the Package Manager and show the packages in the project. Find VR Builder and click [Remove](#). You can then import the new package from the Package Manager.

Note: The demo processes might have changed in the updated version of the package. If you plan to use the demo scenes, ensure to delete the demo processes from the StreamingAssets/Processes folder **before** importing the new package, or the existing process files will not be overwritten.

Upgrading to major versions

Major VR Builder releases are usually not completely compatible with the previous version. Updating to them requires extra care, and is not recommended in production unless you need some of the new features. The specifics can vary depending on the release, and we will provide detailed upgrade guides when necessary. In particular, VR Builder 5 represents a clean cut with the past as it introduces support for Unity 6, XRI 3 and URP. Currently, it is not possible to automatically upgrade a project created with VR Builder 4 to version 5. If you need to do so, [contact us](#) and we will support you with the matter.

Quick Start

You can get a first impression of VR Builder and its features by accessing the provided demo scene. This sample scene contains a pre-built process that showcases some of the interactions provided in VR Builder.

You can automatically open the demo scene at the end of the setup wizard, or use the shortcut in

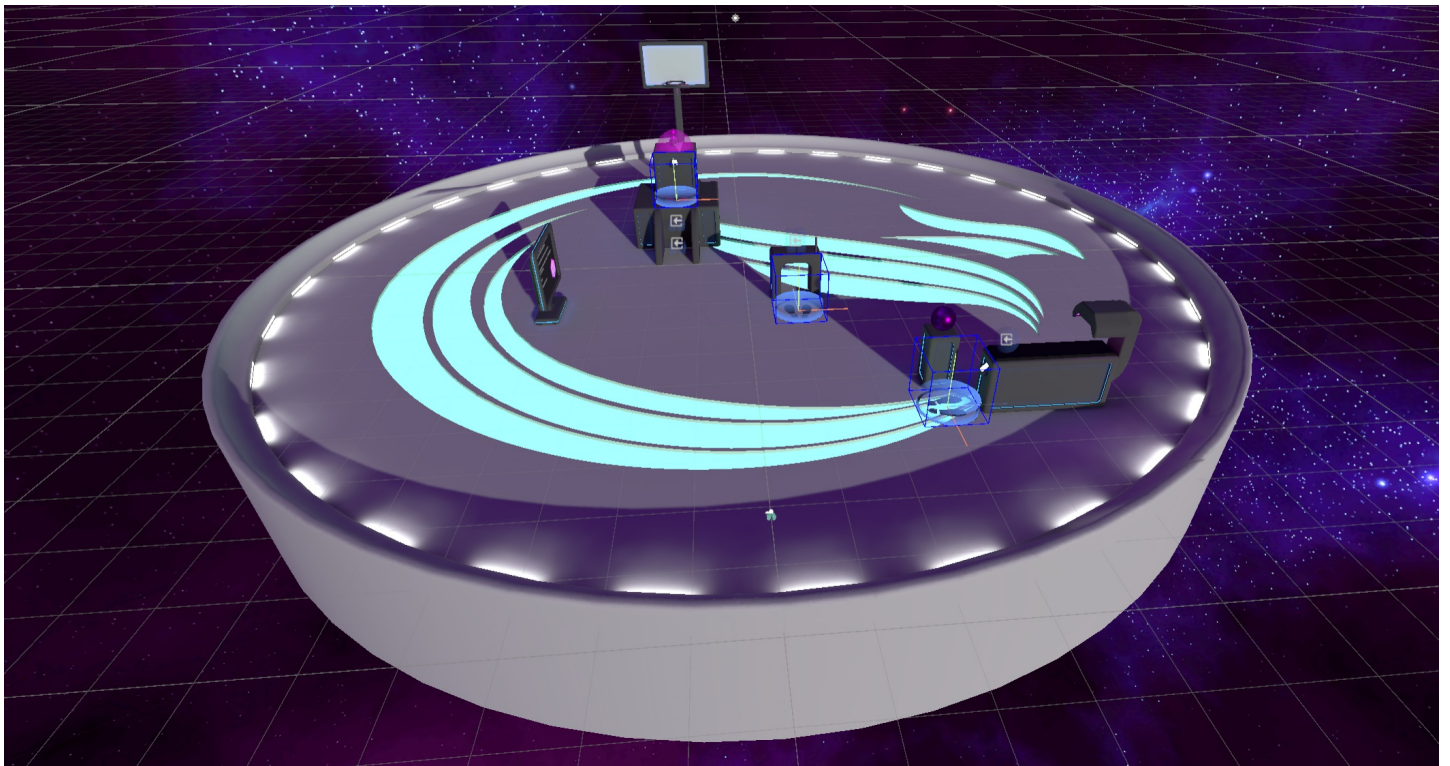
`Tools > VR Builder > Demo Scenes > Core`. Note that, while the scene can be found and opened from disk, it is necessary to use one of the above methods at least once in order to automatically copy the process file to the StreamingAssets folder, where VR Builder processes are saved.

Demo Scene Overview

The demo scene showcases how to assemble a process with the building blocks included in VR Builder. More building blocks and features are available as separate add-ons.

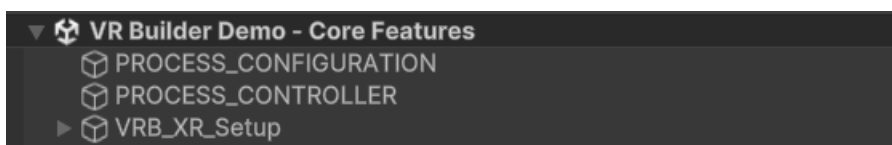
These building blocks are either conditions or behaviors. Conditions check if the user or the world is in a certain state, and behaviors modify the world state when activated.

The process in the demo scene is linear, and will guide the user through different steps. To try out the demo scene, ensure audio volume is up, or you won't be able to hear the spoken instructions!



Demo Scene Hierarchy

In the hierarchy there are three game objects in parentheses.



They are automatically added to every VR Builder scene.

- `PROCESS_CONFIGURATION` allows to select the process for the current scene from a list of processes saved in the project.
- `PROCESS_CONTROLLER` defines some parameters for processes in this scene like the spectator input bindings and camera.
- `VRB_XR_Setup` is the VR rig used by the default XR interaction component. If you are using a different interaction component, for example to use VR Builder in conjunction with [Interhaptics](#) or [VRIF](#), you might see a different rig here.

By looking at the other objects in the scene, we can see that some have a `Process Scene Object` component and possibly some "property" component. A `Process Scene Object` has a unique object ID that can be accessed by the process. Properties define how the process can interact with the object. For example, a `Grabbable Property` will let VR Builder recognize if an object is being grabbed. Adding a `Grabbable Property` to an object will automatically make it a `Process Scene Object` and add a few components so you can interact with the object in VR.

If these properties are not added manually you will usually be prompted to add them automatically when building the process of your VR application.

Customizing the Rig

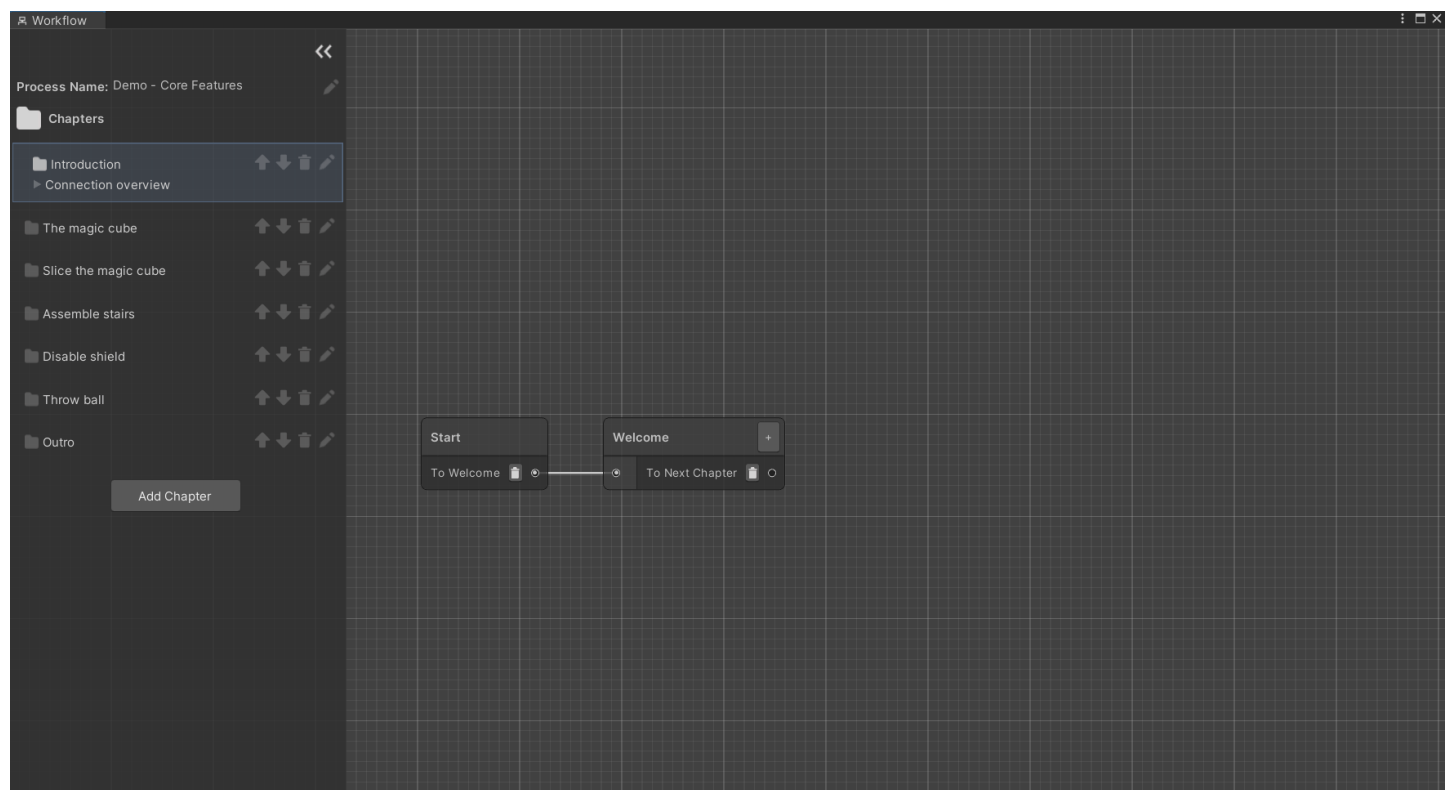
When creating a VR Builder scene, the default rig is created directly in the scene and can be edited or replaced like any game object. If you plan to use the same rig in multiple scenes, just create a prefab of it and manually replace the default rig.

The only requirement every VR Builder rig has, independent of the interaction system, is that it must contain a `User Scene Object` component. This component identifies the rig as the user, and is usually placed on the root of the rig. It should reference the head, hand and base transforms, so that VR Builder can access those positions when needed. If left empty, it will attempt to find the head by itself by looking for the camera's transform.

It is also possible to add other `Process Scene Object`s on the rig in order to use hands, backpacks, toolbelts and so on in behaviors and conditions, depending on the use case.

Process Editor

The process editor lets you design the process of your VR application. You can open the process editor from `Tools > VR Builder > Process Editor` or `Window > VR Builder > Process Editor`. The process editor for the demo scene should look like this.



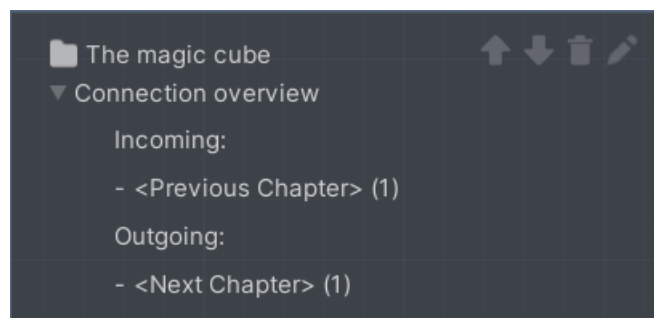
Chapters view

On the left, there is a list of chapters. Every chapter is a separate section of the process. They are useful to separate a process in its logical steps and avoid too much clutter in a single graph.

You can click on the different chapters to visualize the corresponding graphs.

Next to the chapter name, there are icons that allow you to move the chapter up and down in the list, rename it or delete it.

Underneath, you can see the **Connections breakdown** foldout. Expand it to see incoming and outgoing connections for the current chapter. That is, which chapters lead here and to which chapter it is possible to go from this one. The number next to each connection represents the amount of steps that connect to the chapter. When "Next Chapter" or "Previous Chapter" is listed as a connection, it means the connection is implicit: a path ends with an empty transition, which by default ends the current chapter and starts the next one in order.

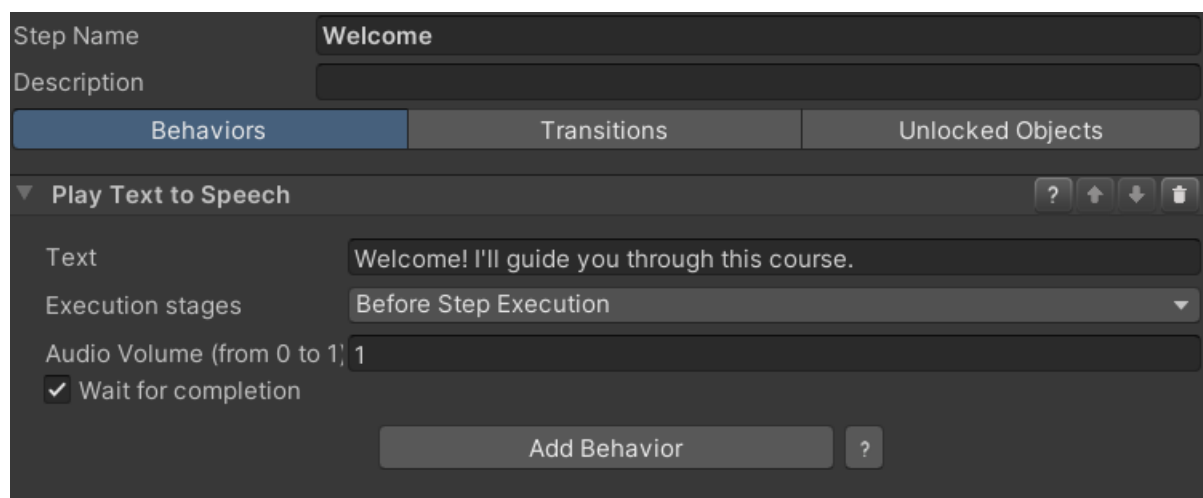


The demo scene is linear, meaning that each chapter will lead directly to the next and the connection overview only contains implicit connections, but it is possible to create more complex processes that don't follow the chapter list linearly.

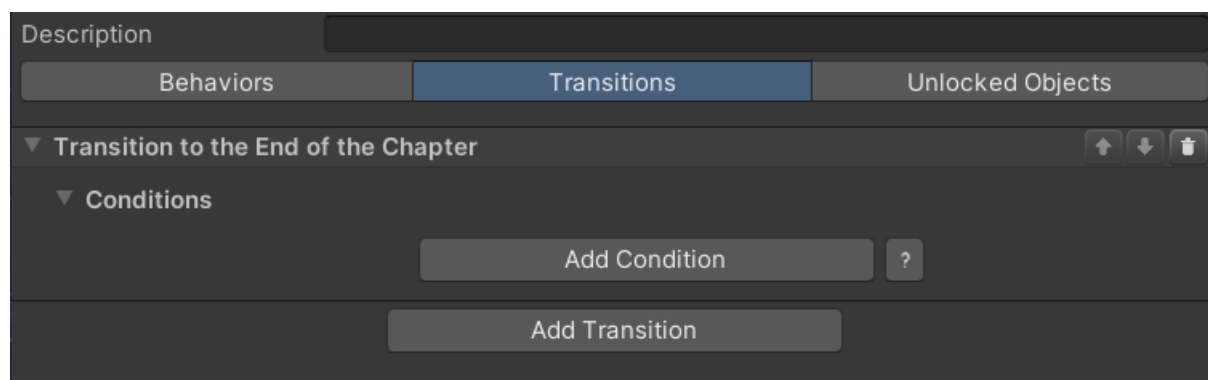
Graph view

On the right, there is a graphical representation of the current chapter. Every node is called a **Step**. Every step can include a number of **Behaviors** which can happen when the node is triggered or before leaving it. In the demo scene, those are mostly text to speech instructions and animations. A step can have as many exit points, called **Transitions**, as needed. Every transition can list a number of **Conditions** which, if fulfilled, make the transition valid.

Select the "Welcome" node in the first chapter. This will open the Step Inspector. The window should look like the following.



The only behavior is a text to speech instruction that will be triggered when the node is entered. Click on the "Transitions" tab.



There is a single transition. A step can have multiple transitions, each leading to a different step. In this case, the transition is connected to no other step, so it will end the chapter. The next chapter, "The magic cube", will then start.

Transitions can include conditions. If they do, they will trigger only when the attached conditions are completed. This transition has no conditions, so it will trigger immediately after the current step has ended, without any input from the user.

We encourage you to investigate the other nodes to understand how the demo scene is built.

Step Nodes

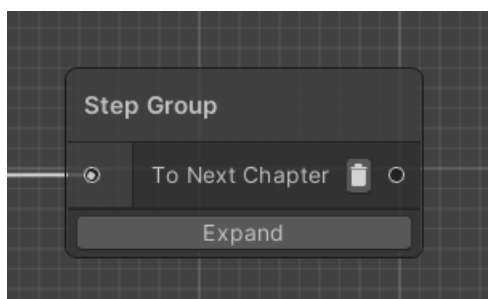
You can create a node by right clicking anywhere in the graph and selecting **New**, then the type of node you want to create. There are four types of node available in VR Builder core:

Step

This is the default step node, the main building block for your process. By default, it is empty. This means that nothing will happen, and the execution will immediately proceed to the next node, if present. You will need to add behaviors and conditions to it in the **Step Inspector** in order to customize it and build your process logic.

Step Group

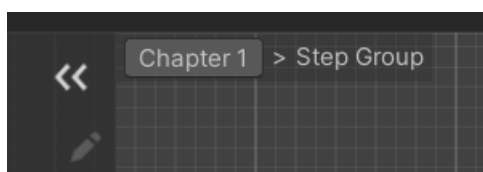
This node doesn't let you set conditions and behaviors, but instead can be expanded in a new node graph. It can be populated with other step nodes and act as a "sub-chapter" with some self contained logic. This can help keeping the process tidy.



You can access the node's graph by clicking the **Expand** button or double clicking on the node itself. There are also context menu options for expanding the node or ungrouping it - that is, replacing it in the main graph with the logic it contains.

This node only has one entry and one exit point. This means that after the contained logic has ended executing, the process will always continue executing from the exit transition of the group node.

If you are in a step group graph, it will be indicated on the top left of the process editor.



You can click on a parent to return to it. For example, clicking on "Chapter 1" will get you back to the main chapter graph.

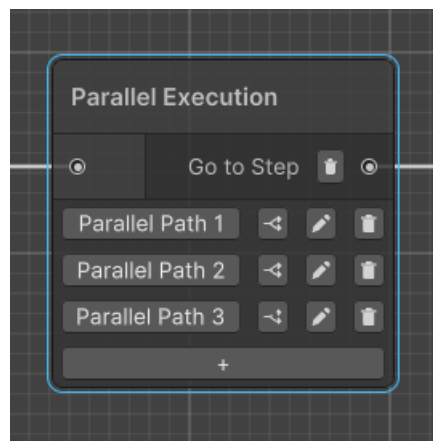
You can also create a group by selecting a sequence of nodes, right clicking and selecting **Make group**. Since the step group node only has one input and one output, this works best when selecting linked nodes only. Edge cases are resolved as follows:

- If there are two or more inbound connections in the group, all will lead to the group's input. The first valid node will be chosen as starting step for the group, while the others will have their connection severed.
- All outgoing connections will be deleted, meaning that the process will continue from the output of the group node after the group has processed. This means that if the selected nodes lead to multiple other nodes, now they will all go through the group's output.
- The step group output will be connected to the previous target of the first valid grouped node. Other external targets in grouped nodes are ignored, which means that when the group ends it will always go to the same following node.

If you encounter one of these edge cases, make sure to review your process logic after grouping, as it may have changed.

Parallel Execution

The parallel execution node lets you execute two or more step sequences at the same time. Execution will continue to the next node once all parallel sequences have completed.



Clicking on a Parallel Path button will open a new graph where the path can be edited. This is very similar to a step group, with the difference that there can be multiple parallel paths and they are all executed at the same time. Like with step groups, it is possible to return to the main process by clicking the root chapter on the top left of the process editor.

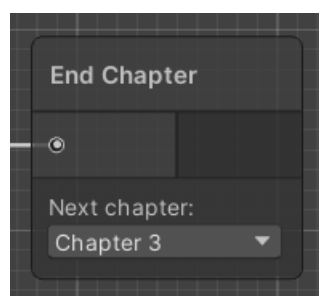
The buttons next to a parallel path let you make the path optional, rename it or delete it. Non-optional paths are displayed with the continuous arrow icon, while the interrupted arrow icon denotes a path that is optional.

When all non-optional paths have finished execution, the node ends execution as well. All optional paths are immediately interrupted. It can be useful to create optional paths to display looping animations, recurring hints and so on - they can even be endless loops without an exit point, as they will be interrupted anyway when the step ends.

The "+" button at the bottom lets you add more parallel paths. There is no theoretical limit to the number of paths in a parallel execution node, but a too high number might impact performance.

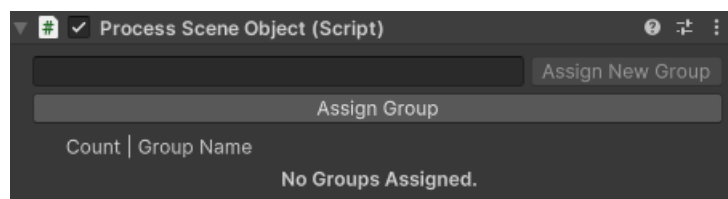
End Chapter

You can use this node as the last node on a sequence. It will immediately end the current chapter and start a new specified chapter, which can be selected from a drop-down list. This is useful to move through the chapters in a non-linear fashion. Note that you are not required to use this node for linear processes, as a chapter will automatically end when an empty transition is reached. In that case, the process will simply proceed to the following chapter.



Process Scene Objects

The `Process Scene Object` component acts as a bridge between the VR Builder process and a Unity game object. If the process needs to observe or interact with a game object, it does so through this component. This means that every game object referenced in the process is required to have this component. If a more detailed interaction is needed, property components can be added to provide VR Builder more control on the object. For example, adding a `Grabbable Property` component will make the object grabbable, and VR Builder will be able to check if the object has been grabbed.



The `Process Scene Object` generates a hidden object ID which identifies the object internally in the VR Builder process.

In addition, it is possible to associate an arbitrary number of groups to every scene object. Some behaviors and conditions can interact with unspecified objects within a certain group rather than a specific object with a given object ID.

You can select and assign an existing group from the list, or create and directly assign a new group. You can unassign a group from an object by clicking the X button next to it. Groups are stored on a per-project basis and can be created, edited or deleted from `Project Settings > VR Builder > Scene Object Groups`.

It is possible to edit multiple Process Scene Objects at the same time to add or remove groups in bulk. When multiple objects are selected, all groups on all objects are listed.

If a group is present only on some of the selected objects, it will be displayed in *italics*. A default text style means that the group is present on all selected objects.

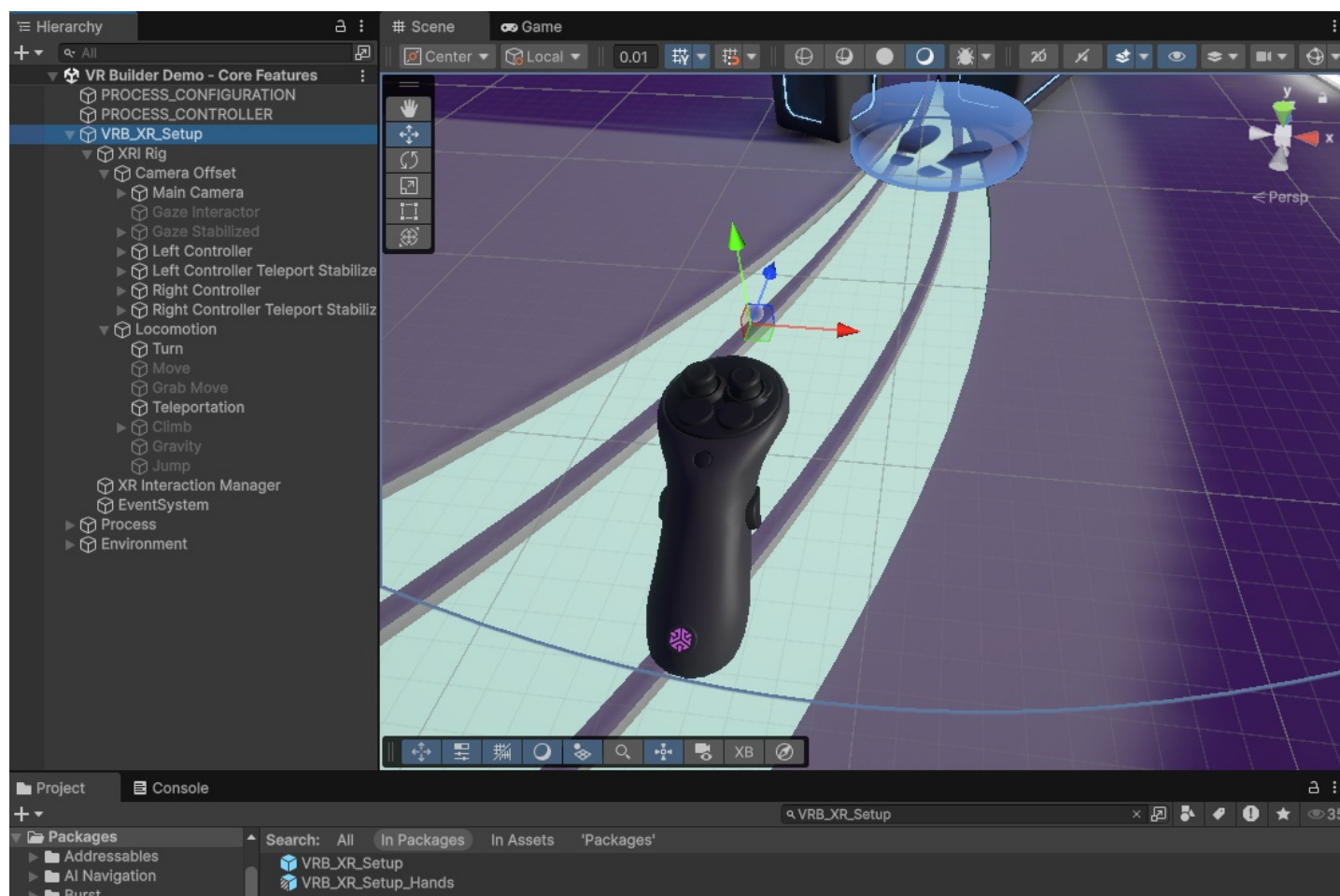
XR Rig

The XR Rig is a game object that anchors the user's head mounted display, controllers, or user's hands when using hand tracking within the virtual space. Essentially, it represents the user in the scene. On the rig, you can configure which types of locomotion and interaction are available to the user.

VR Builder XR Rig

When creating a new scene with the VR Builder Scene Setup Wizard, you get a `VRB_XR_Setup` game object automatically added to the scene. `VRB_XR_Setup` is the VR Builder XR Rig and is built upon the suggested standards set by the Unity XR Interaction Toolkit. Therefore, `VRB_XR_Setup` is very similar to the `XR Origin (XR Rig)` found in the XR Interaction Toolkit samples but has some additional VR Builder-specific scripts added.

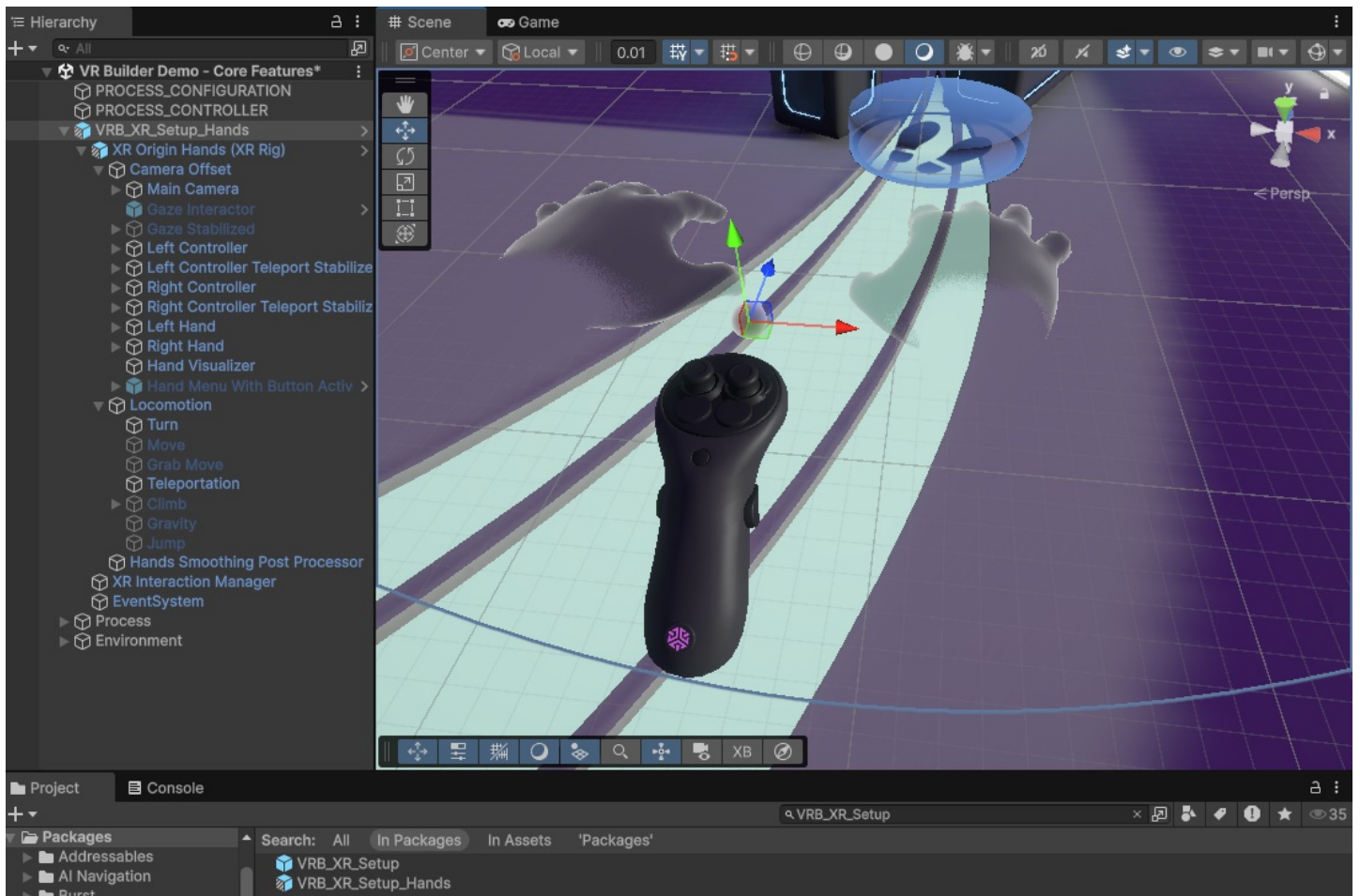
The `VRB_XR_Setup` game object is a fully unpacked copy of the prefab with the same name. If you are familiar with Unity's prefab workflows you can use the prefab or better your own prefab variant of it.



VR Builder XR Rig with Hand Tracking

VR Builder also includes an XR Rig that supports hand tracking. To use it, search for `VRB_XR_Setup_Hands` in the VR Builder Package and replace the `VRB_XR_Setup` game object with the `VRB_XR_Setup_Hands` prefab. Depending on your preferences, you can keep the `VRB_XR_Setup_Hands` prefab (see image below) or completely unpack it.

Just like the `VRB_XR_Setup`, the `VRB_XR_Setup_Hands` is built on top of the XR Interaction Toolkit samples and aligns with these standards. For example, it supports automatic switching between hands and controllers. During runtime, the following logic is used for determining which modality is active: If a hand begins tracking, this component will switch to the hand group of interactors. If the player wakes the motion controllers by grabbing them, this component will switch to the motion controller group of interactors once they become tracked. While waiting to activate the controller GameObject while not tracked, both groups will be deactivated. For the exact implementation behavior or if you want to modify it, refer to the `XRIInputModalityManager` inside the rig.

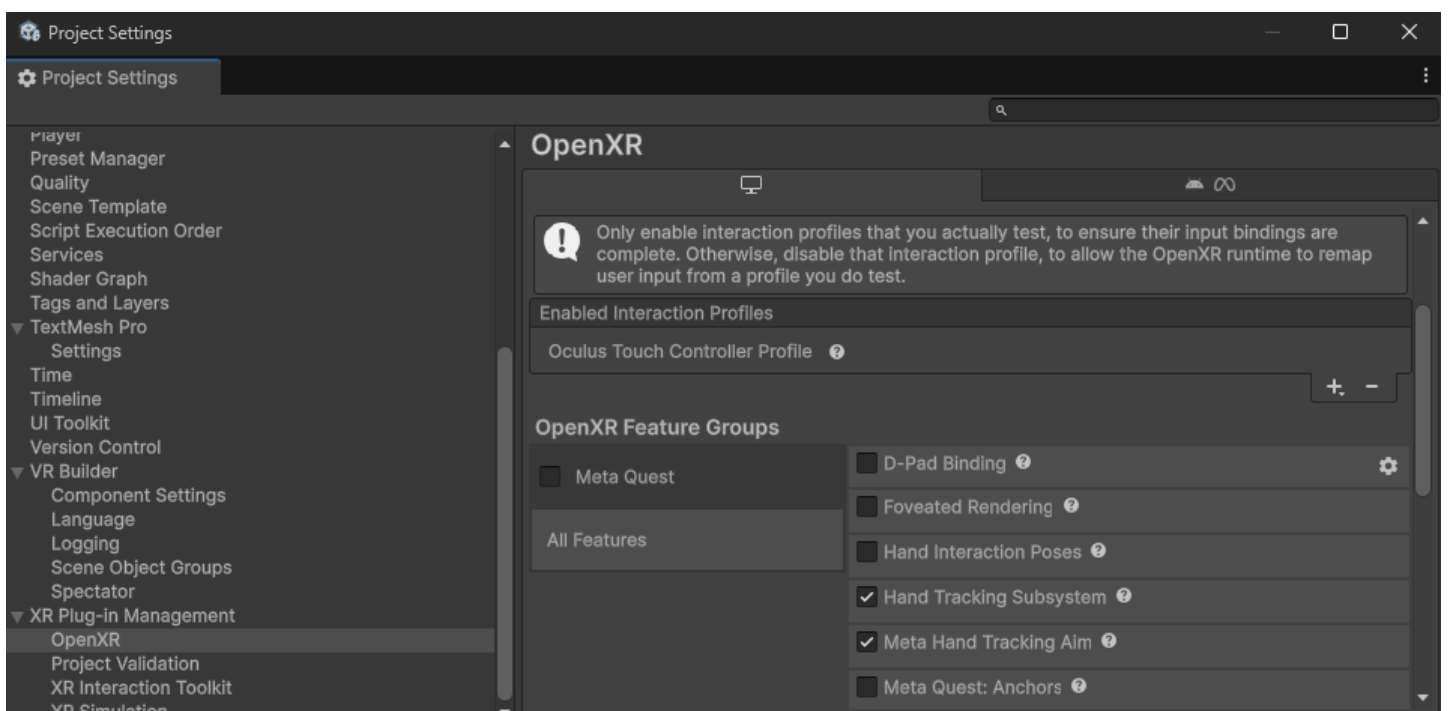


Hand Locomotion

If users need to move with hand tracking enabled, there is the built-in Grab Move locomotion type. Other ways of locomotion (e.g., teleportation via a hand gesture) are possible but require custom implementation effort.

Troubleshooting

If your hands do not show up, ensure that the OpenXR Feature Groups `Hand Tracking Subsystems` and `Meta Hand Tracking Aim` are enabled in the OpenXR page in the Project Settings.

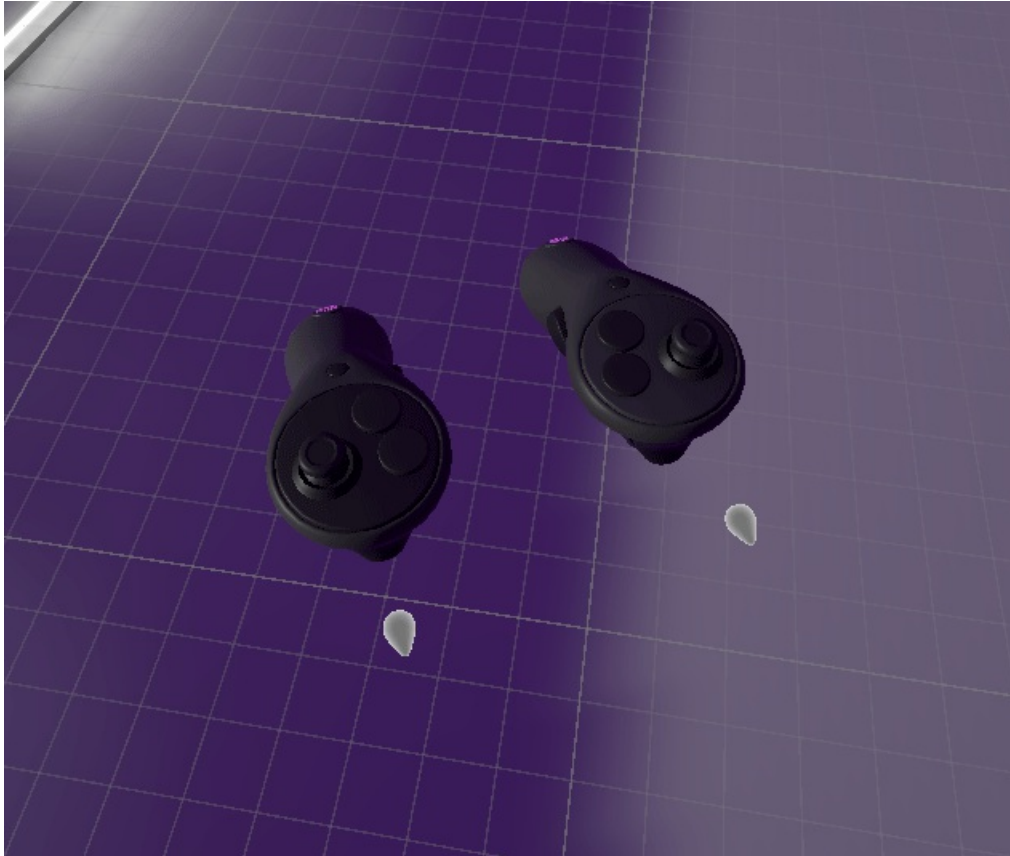


Controller Visualization

By default, the VR Builder rig visualizes the VR controllers as generic controllers. With some small modifications on the rig, you can change this to show animated hands that react when pressing the grip or trigger button on your controller.

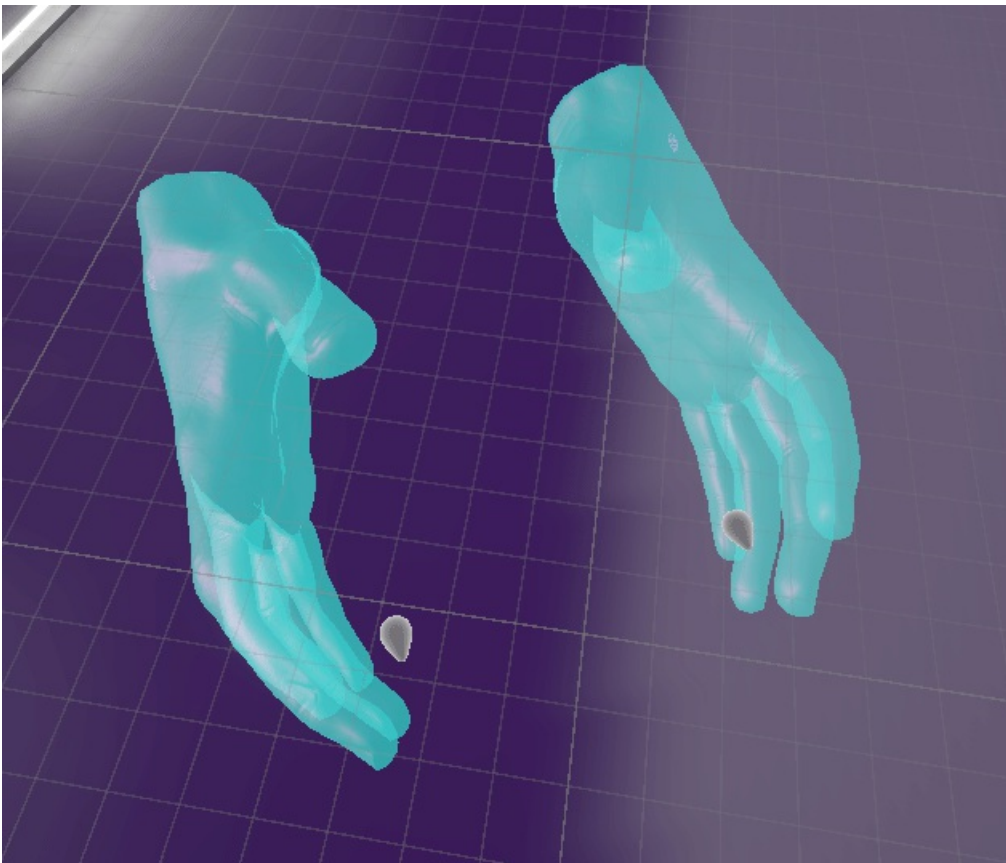
Animated Controllers

To show or hide the controller visualization, navigate to `Left Controller Visual` and `Right Controller Visual` inside the rig and enable or disable them.



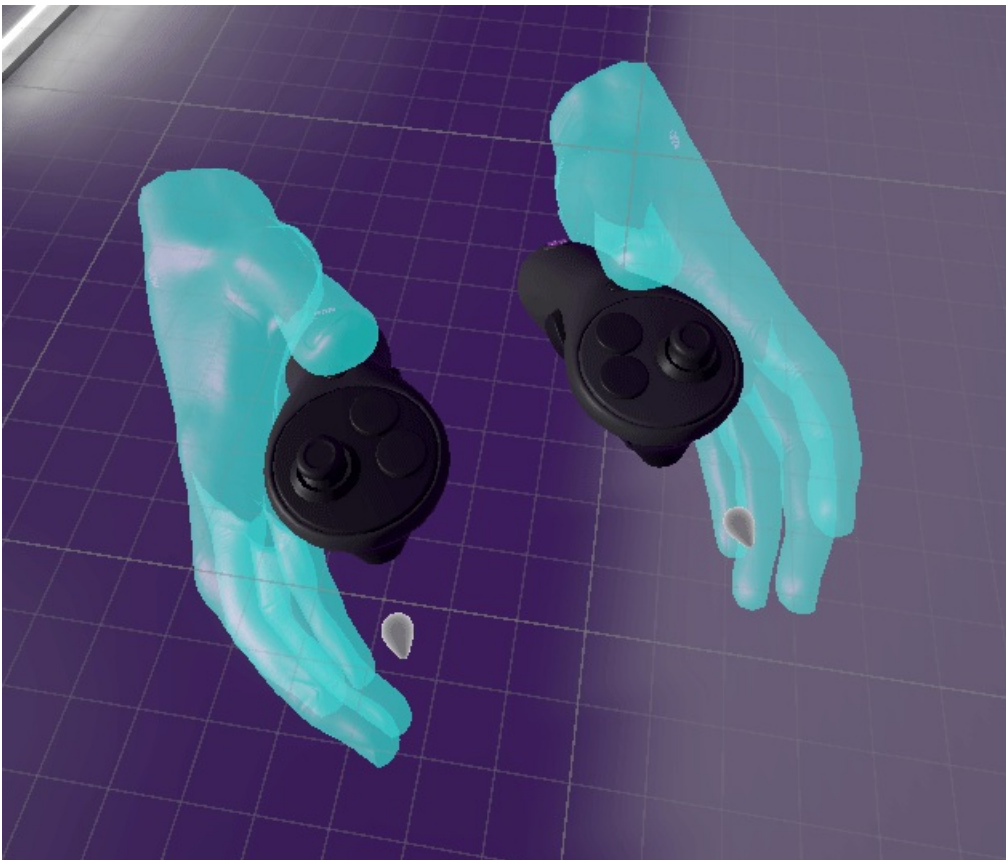
Animated Hands

To show or hide the hands visualization, navigate to `Left Hand Visual` and `Right Hand Visual` inside the rig and enable or disable them.



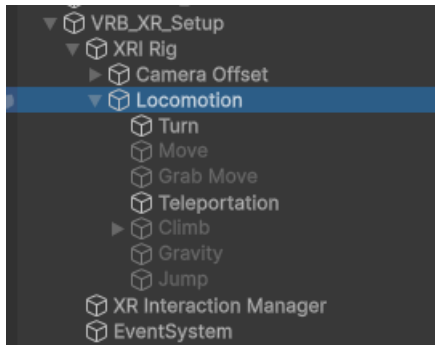
Dual Visualization

It is possible to show both controllers and hands at the same time by having both visualizations active. However, our priority was that the animated hands work well on their own and interact smoothly with the grey `Poke Point` and the teleport ray. Because of this, the hand and finger positions are not accurately mapped to the controller buttons.



Locomotion

By default the rig is configured for locomotion types that minimize discomfort. These are movement by teleportation and rotation with snap turn. You can modify the locomotion styles supported by your rig by enabling or disabling the child objects of the `Locomotion` object on the rig.



General Documentation

For detailed explanations on all locomotion styles supported please refer to the XR Interaction Toolkit documentation. As of 04.08.2025 VR Builder uses XR Interaction Toolkit [version 3.1.2](#) as the source of the rig.

For a more in-depth guide on locomotion in VR we suggest reading the excellent guide at [Meta's locomotion design guidelines](#).

Setup Teleportation in VR Builder

Teleportation requires the user to push the right thumbstick forward then release it while pointing at a teleportable surface or anchor. Most of the setup is handled automatically by VR Builder, but a few steps are required for it to work properly.

Layer Configuration

It is recommended that teleportation use a layer different from normal interactions. By default, VR Builder uses the `Teleport` raycast layer for controller raycasts and the `Teleport` XRI interaction layer for teleport actions. When you import VR Builder, it attempts to create both layers at index 31 of their respective layer arrays.

Interactors and interactables must be configured with these layers to function. In a new scene, the default rig's teleport interactors are already set up to raycast and interact on these layers.

Anchor and Area Setup

When creating a teleportation anchor or area, use the **Teleportation Anchor (VR Builder)** or **Teleportation Area (VR Builder)** components instead of the XRI versions. Each component has a button in its inspector that configures compatible settings, including layer masks. The **Teleportation Anchor** also offers additional options:

- **Set Default Teleportation Anchor:** Creates a default anchor by adding a collider to the game object and a graphical representation of the anchor (a feet silhouette inside a circle).
- **Add Snap Volume:** Configures the anchor so that the teleportation ray snaps to it when it's in the vicinity.
- **Add Interaction Affordance:** Adds feedback when the user interacts with the anchor.
- **Add Proximity Entry:** If this is enabled, the user will be considered teleported at the anchor even if they get close by walking or stick locomotion. This can be relevant if the VR Builder process includes a [Teleport condition](#) to that anchor.

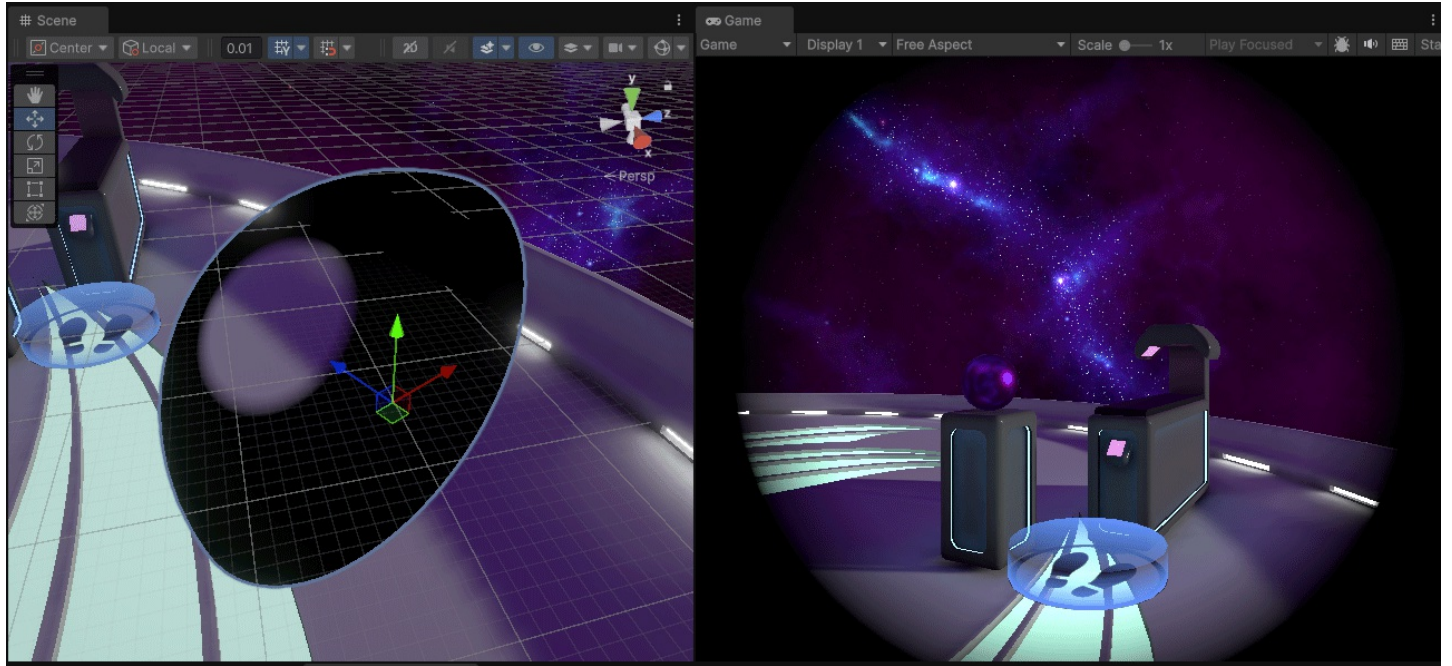
Troubelshoting

Teleportation generally works out of the box. If you encounter issues—such as after editing an old scene or moving the XR Teleport layer—you can reset all layers on the rig, anchors, and areas. To do this, select:

`Tools > VR Builder > Developer > Configure Teleportation Layers`. Be aware that this will overwrite existing layer masks and assign the `Teleport` layer to all affected objects.

Tunneling Vignette

The tunneling vignette improves comfort during continuous movement. By default, the vignette is configured to activate whenever any continuous [Locomotion](#) method is in use on the rig. Note that continuous locomotion methods are disabled by default on the VR Builder rig, so the vignette will only be used if you manually enabled them.



Configure Tunneling Vignette

To configure it, search for the `TunnelingVignette` game object in the rig. For detailed explanations of the tunneling vignette settings, refer to the [XR Interaction Toolkit documentation](#).

Behaviors

Behaviors are used to make something happen in the process. Behaviors can be as simple as giving instructions and hints, making new objects visible, or animating an object in the scene. What they have in common is that something happens without requiring an intervention by the user. This section lists the behaviors included in VR Builder.

Move Object

Description

The Move Object behavior animates the `Object` to move and rotate (no scaling) to the position and rotation of the `Final Position Provider` in the time specified in the `Duration (in seconds)` parameter.

Note: If `Object` was affected by gravity before, it will continue to be affected after this behavior.

Configuration

- **Object**

The `Process Scene Object` to be moved and rotated (no scaling).

- **Final position provider**

The `Process Scene Object` used as the position provider object. It should be placed at the destination position and rotation.

- **Animation duration (in seconds)**

Time in seconds the animation takes to move and rotate `Object` to the `Final position provider`.

Example

Duration (in seconds) = 1.3

- **Animation curve**

Lets you tweak the speed of the animation over time.

Scale Objects

Description

This behavior scales the target objects over time towards a specified value.

Configuration

- **Target Objects**

The `Process Scene Objects` to be scaled. All objects will be scaled at the same time.

- **Target Scale**

The scale on the three axes that will be applied to the target objects.

- **Animation duration (in seconds)**

Time in seconds it takes for the target objects to reach the target scale.

Example

Duration (in seconds) = 1.3

- **Animation curve**

Lets you tweak how quickly the scale changes over time.

Disable Objects

Description

The Disable Objects behavior makes the selected `Objects` invisible and non-interactive until they are specifically set back to *"enabled"* in a future step. Put into Unity terms, it deactivates the selected game objects.

Configuration

- **Objects**

The `Process Scene Objects` to be disabled.

Enable Objects

Description

The Enable Objects behavior makes the selected `Objects` visible and interactive until it is specifically set back to *"disabled"* in a future step. Put into Unity terms, it activates the selected game objects.

Configuration

- **Object**

The `Process Scene Objects` to be enabled.

Disable Components

Description

The Disable Components behavior disables all components of the specified type on the given `Process Scene Objects`.

Configuration

- **Object**

The `Process Scene Objects` the component is on.

- **Component type**

A drop-down list allowing to select the component type that will be disabled.

- **Enable at end of step**

If checked, the components will be enabled again at the end of the step.

Enable Components

Description

The Enable Components behavior enables all components of a specified type on the given `Process Scene Objects`.

Configuration

- **Object**

The `Process Scene Objects` the component is on.

- **Component type**

A drop-down list allowing to select the component type that will be enabled.

- **Disable at end of step**

If checked, the components will be disabled again at the end of the step.

Start Particle Emission

Description

The Start Particle Emission behavior starts particle emission on the selected objects. It can be useful for effects that should begin only when a process step is reached, such as smoke, sparks or ambient particles.

Configuration

- **Targets**

The `Process Scene Objects` that should start emitting particles. Each target needs a `Particle System Property` component.

- **Execution stages**

By default, steps execute behaviors in the beginning, in their activation stage. This can be changed with the `Execution stages` dropdown menu:

- `Before Step Execution`: The step invokes the behavior during its activation.
- `After Step Execution`: Once a transition to another step has been selected and the current step starts deactivating, the behavior is invoked.
- `Before and After Step Execution`: Execution at activation and deactivation of a step.

Stop Particle Emission

Description

The Stop Particle Emission behavior stops particle emission on the selected objects. This is useful when an effect should end at a specific point in the process.

Configuration

- **Targets**

The `Process Scene Objects` that should stop emitting particles. Each target needs a `Particle System Property` component.

- **Execution stages**

By default, steps execute behaviors in the beginning, in their activation stage. This can be changed with the `Execution stages` dropdown menu:

- `Before Step Execution`: The step invokes the behavior during its activation.
- `After Step Execution`: Once a transition to another step has been selected and the current step starts deactivating, the behavior is invoked.
- `Before and After Step Execution`: Execution at activation and deactivation of a step.

Unsnap Object

Description

The Unsnap Object behavior unsnaps a snapped object from a snap zone. This can be useful in case the object needs to be further manipulated by the process.

Configuration

Either the object or the snap zone can be left null. This will result in either the object unsnapping from any snap zone it is in, or in the unsnapping of whatever object is snapped to the specified snap zone.

If both are specified, the unsnap will occur only if the specified object is snapped to the specified snap zone.

- **Object to unsnap**

The `Process Scene Object` to unsnap.

- **Snap zone to unsnap**

The `Snap Zone` from which the object will be unsnapped.

Play Audio File

Description

The Play Audio File behavior plays an audio clip loaded from the `Resources` folder in your project's asset folder. VR Builder supports all audio file formats supported by Unity, which are:

- aif
- wav
- mp3
- ogg

Configuration

- **Resources path/Key**

Insert the relative file path from the Resources folder. Please note that you have to omit the file extension, as shown in the example. You can find more information about this on <https://docs.unity3d.com/ScriptReference/Resources.Load.html>.

If localization is enabled, this should instead be a key in the localization table associated with the process, which contains the localized resources path.

Example without localization

File to be played: `Assets/.../Resources/Sounds/click-sound.ogg`

Text/Key: `Sounds/click-sound`

Example with localization

File to be played: `Assets/.../Resources/Sounds/click-sound.ogg`

Text/Key: `click-sound-key`

Localization table translation: `Sounds/click-sound`

- **Volume**

The volume at which the audio should be played.

- **Execution stages**

By default, steps execute behaviors in the beginning, in their activation stage. This can be changed with the `Execution stages` dropdown menu:

- `Before Step Execution`: The step invokes the behavior during its activation.
- `After Step Execution`: Once a transition to another step has been selected and the current step starts deactivating, the behavior is invoked.
- `Before and After Step Execution`: Execution at activation and deactivation of a step.

- **Wait for completion**

By default, the step waits for the audio file to finish. If you want the step to interrupt the audio in case the step is completed, uncheck this option.

Note: this might lead to an audio file not even being started, in case the step ends immediately.

Play TextToSpeech Audio

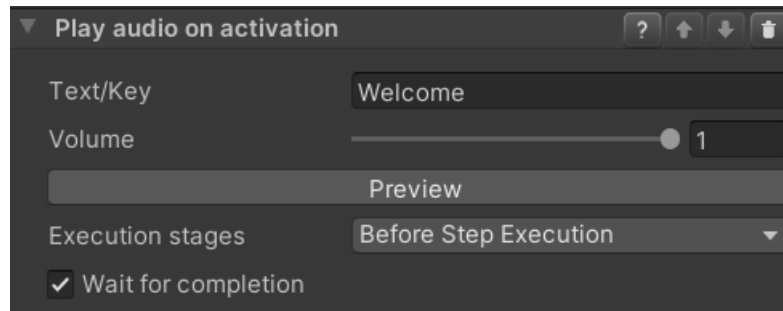
Description

The Play TextToSpeech Audio behavior uses a synthesized voice to read text. It supports localized text through Unity's Localization package.

By default, VR Builder works with a single language. In `Project Settings > VR Builder > Language` you can specify which language will be used by the TTS engine.

VR Builder can also be configured to use the Localization package from Unity to provide localized text. The Project Setup Wizard can guide you through the steps, which are the same as outlined in the official [documentation](#).

VR Builder will automatically switch to localized mode when a Localization Settings object has been created.



Configuration

- **Text/Key**

If the project is not configured for localization, this field can be used to enter the text that will be spoken in the language configured in the Project Settings.

If localization is configured, meaning that a Localization Settings object has been created, you need instead to enter the key to look for in the localization table associated with the process.

- **Volume**

The volume at which the audio should be played.

- **Execution stages**

By default, steps execute behaviors in the beginning, in their activation stage. This can be changed with the `Execution stages` dropdown menu:

- `Before Step Execution`: The step invokes the behavior during its activation.
- `After Step Execution`: Once a transition to another step has been selected and the current step starts deactivating, the behavior is invoked.
- `Before and After Step Execution`: Execution at activation and deactivation of a step.

- **Wait for completion**

By default, the step waits for the audio file to finish. If you want the step to interrupt the audio in case the trainee completes the conditions, uncheck this option.

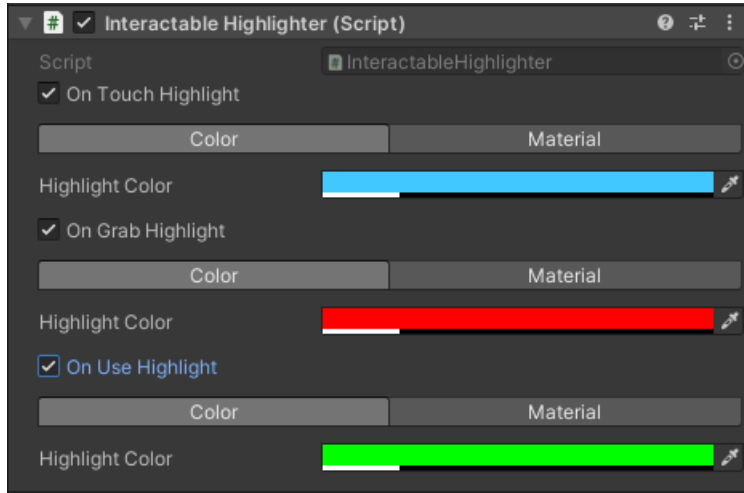
Note: this might lead to an audio file not even being started, in case the step ends immediately.

Highlight Object

Description

The Highlight Object behavior visually highlights the selected object until the end of a step.

For additional highting features you can replace the *Default Highlighter* with a *Interactable Highlighter Script*. Select the highlighted `Object` in the Unity Hierarchy. In the Unity Inspector replace the *DefaultHighlighter* with a *Interactable Highlighter Script*.



You can define the Color and Material for *On Touch Highlight*, *On Grab Highlight*, and *On Use Highlight*. The object will show the highlight color configured in the Highlight behavior by default. As soon as the object is touched it will change to the color configured in *On Touch Highlight*. The same happens when the object is grabbed or used. It will display the configured color in 'On Grab Highlight' or 'On Use Highlight'.

Configuration

- **Color**

Color in which the target object will be highlighted. Colors are defined in the RGBA or HSV color channel. By configuring the alpha (A) value, highlights can be translucent.

- **Object**

The `Process Scene Object` which should be highlighted.

Spawn Confetti

Description

The Spawn Confetti behavior causes confetti to fall above the selected `Object`. It can be useful as visual feedback or celebration for completing a task successfully.

Configuration

- **Spawn Above User**

If checked, the spawn position will be above the user rather than on the specified `Process Scene Object`.

- **Position Provider**

Specifies where the confetti should spawn if not set to spawn above the user.

- **Confetti Machine Path**

Path to the confetti machine prefab, relative to a `Resources` folder. Use the default one or point to your custom confetti machine.

- **Area Radius**

Radius around the position provider in which confetti will be spawned.

- **Duration**

Duration of the visual effect in seconds.

- **Execution stages**

By default, steps execute behaviors in the beginning, in their activation stage. This can be changed with the `Execution stages` dropdown menu:

- `Before Step Execution`: The step invokes the behavior during its activation.
- `After Step Execution`: Once a transition to another step has been selected and the current step starts deactivating, the behavior is invoked.
- `Before and After Step Execution`: Execution at activation and deactivation of a step.

Behavior Sequence

Description

The Behavior Sequence contains a list of child behaviors which will be activated one after another. A child behavior in the list will not be activated until the previous child behavior has finished its life cycle.

Configuration

- **Repeat**

If checked, the behavior sequence restarts from the top of the child behavior list as soon as the life cycle of the last child behavior in the list has finished.

- **Child behaviors**

List of all queued behaviors. Add behaviors to the list using the "*Add Behavior*" button.

- **Wait for completion**

If checked, the behavior sequence will finish the life cycle of each child behavior in the list before it transitions to another step. Even when the "*Repeat*" option is enabled, the execution will transition to the next step after the child behavior list has been completed. Uncheck this option if you want to interrupt the sequence as soon as all conditions of a transition are fulfilled.

Example

Play an audio file after a set time, for example to give the user some delayed hints.

This is a sequence combining a Delay and a Play Audio File behavior. Refer to the documentation for the [Delay behavior](#) and the [Play Audio File behavior](#).

Delay

Description

The Delay behavior completes after the specified amount of time. This step will wait for the duration configured in `Delay (in seconds)`, even when the user fulfills the required conditions to transition to the next step.

Configuration

- **Delay (in seconds)**

Configure the behavior's delay duration in seconds.

Example

Delay (in seconds) = 1.3

Set Parent

Description

The Set Parent behavior parents an `Object` to another one in the Unity hierarchy.

Configuration

- **Target**

The `Process Scene Object` to be parented.

- **Parent**

The new parent for the target object. Note this can be null, in which case the object will be unparented.

- **Snap to parent transform**

If checked, the target object will snap to the same position and rotation as the parent object.

Load Scene

Description

The Load Scene behavior loads a new scene, either additively or not. If a scene is loaded additively, it should not contain a VR Builder process as that could interfere with the process running in the main scene. It still can contain process scene objects which can be interacted with. If a scene is loaded non-additively, this behavior will effectively interrupt the process as the current scene is unloaded and a new one is loaded.

Configuration

- **Scene to load**

Select the scene to be loaded from this dropdown. Note that the scene needs to be added to the Build Settings in order to show in the dropdown. The scene is serialized by path in the process JSON, so moving or renaming it might break this parameter.

- **Load additively**

If checked, the scene will be loaded additively and the process will keep running. If unchecked, the current scene will be unloaded and thus interrupt the current process.

- **Load asynchronously**

If checked, the scene will be loaded asynchronously. This can result in smoother performance while loading. A scene is loaded synchronously in the Start phase of the activating process, while asynchronous loading will happen in the Update phase.

Conditions

Conditions are used to determine which transition is used to exit a step. Transitions are evaluated top to bottom, and the first valid one will be selected. To be valid means to have only fulfilled conditions or no conditions at all. A condition usually requires the user's intervention, for example grabbing an object. However this is not always the case: conditions like the timeout condition will trigger regardless of the user's activity. There may also be cases in which other factors in the environment determine if a condition is fulfilled. Conditions need to be active in order to be fulfilled. As soon as a step is active, all containing Conditions are active as well. This section lists the conditions included in VR Builder.

Move Objects in Collider

Description

The Move Objects in Collider condition is fulfilled when at least `Required object count` of the configured `Objects` are inside the specified `Collider` for the required amount of time (`Required seconds inside`) while this condition is active.

Configuration

- **Objects**

The `Process Scene Objects` that should be inside the collider.

- **Collider**

The `Process Scene Object` with the destination collider. Make sure that a collider is present and that the option `Is Trigger` is enabled.

- **Required seconds inside**

Set the time in seconds that the `Objects` should stay inside the `Collider`.

- **Required object count**

Set how many `Objects` should stay inside the `Collider` at the same time. If this is set to **0** (or lower), the condition is always fulfilled. If `Required object count` is **higher than** the configured `Process Scene Objects` count, the condition is **unfulfillable**.

Remarks

- `Objects` do not need colliders for this condition to work.
- If the `Objects` need to be grabbable themselves, they need the `Grabbable Property` and a collider component. The collider defines the area where the user can grab the object.
- An `Object` counts as inside the collider if its transform position is inside the `Collider`. This means a 3D object can count as inside even if it is only partially inside the `Collider`. You can have one grabbable object with multiple child `Objects` to ensure that the whole object is inside, for example, on a cube with an `Object` on each corner.
- The implementation uses `Collider.ClosestPoint` with the `Object` transform position. If the closest point equals the queried position (within floating-point tolerance), the point is considered inside.

Object Nearby

Description

The Object Nearby condition is fulfilled when the `Object` is within the specified `Range` of a `Reference object`.

Configuration

- **Object**

The `Process Scene Object` that should be in the radius of the `Reference Object`.

- **Reference Object**

The `Process Scene Object` from which you want to measure the distance.

- **Range**

In this field, you can set the maximum distance between the *Object* and the *Reference object* required to fulfill this condition.

- **Required seconds inside**

In this field, you can set the time in seconds the `Object` should stay within the radius `Range` of the `Reference Object`.

Grab Object

Description

The Grab Object condition is fulfilled when the user grabs any of the `Objects`. The condition is also fulfilled if the user already grabbed any of Objects before the step was activated, that is, if the user is already holding the specified object.

Configuration

- **Objects**

The `Process Scene Objects` to grab. The objects need to have the `Grabbable Property` and a collider component configured. The collider defines the area where the user can grab an object.

- **Lock settings**

If the `Inherit Scene Object Lock State` is checked on the `Grabbable Property`, the `Process Scene Object` will inherit the lock state of its parent `Process Scene Object`.

If the `Is Always Unlocked` is checked on the `Grabbable Property`, the `Process Scene Object` will not be locked by the VR Builder process at runtime. It is still possible to lock the `Grabbable Property` manually.

Release Object

Description

The Release Objects condition is fulfilled when all of the `Objects` are released by the user's controller. If the user is not already holding any of the specified objects in hand while this condition is active, it is fulfilled immediately.

Configuration

- **Objects**

The `Process Scene Objects` to release. The objects need to have the `Grabbable Property` and a collider component configured.

- **Lock settings**

If the `Inherit Scene Object Lock State` is checked on the `Grabbable Property`, the `Process Scene Object` will inherit the lock state of its parent `Process Scene Object`.

If the `Is Always Unlocked` is checked on the `Grabbable Property`, the `Process Scene Object` will not be locked by the VR Builder process at runtime. It is still possible to lock the `Grabbable Property` manually.

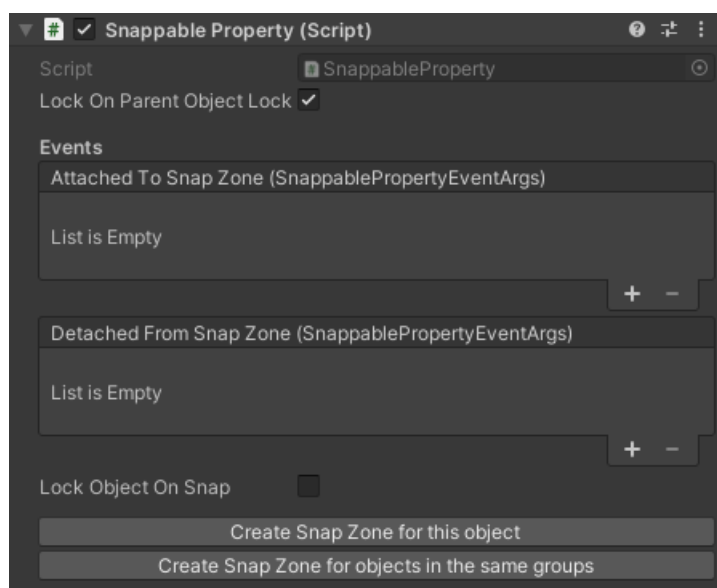
Snap Object

Description

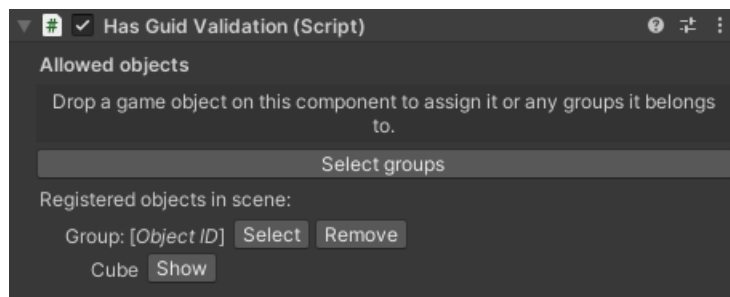
The Snap Object condition is fulfilled when any of the `Objects` is released into the `Zone to snap into`, which means the collider of any of the Objects and collider of the Zone overlap. Adapt the collider size of the snap zone to increase or decrease the area where the user can release an `Object`. Increasing the collider size of the snap zone decreases the required *snap* precision and simplifies the user's interaction in VR. After the user releases an `Object`, this is moved to the snap zone's `SnapPoint`. To adjust this position, change the position of the SnapPoint child object of the `Zone to snap into` object.

- **Snap Zone Generator**

For any snappable object you can generate a snap zone that can snap this object and can be used as a `Zone to snap into`. To do so, navigate to the `Snappable Property` in Unity's Inspector. You will see two buttons. Clicking on the button `Create Snap Zone for this object`, you will create a snap zone that only accepts this exact object. Clicking on `Create Snap Zone for objects in the same groups` will instead create a snap zone that accepts every object that shares at least a group with the current object. Note that the current object will still determine the shape of the snap zone's ghost object.



If you want a specific snap zone behavior, like for example accepting only one group of the source object instead of all of them, you'll have to manually change that. Select the snap zone and find the `Has Guid Validation` component. If you manually created your snap zone, you might have to add this component manually as well. Add to the list the objects you want to be valid for this snap zone. You can reference single objects by object ID, or entire categories of objects via groups.



- **Manual Snap Zone Creation**

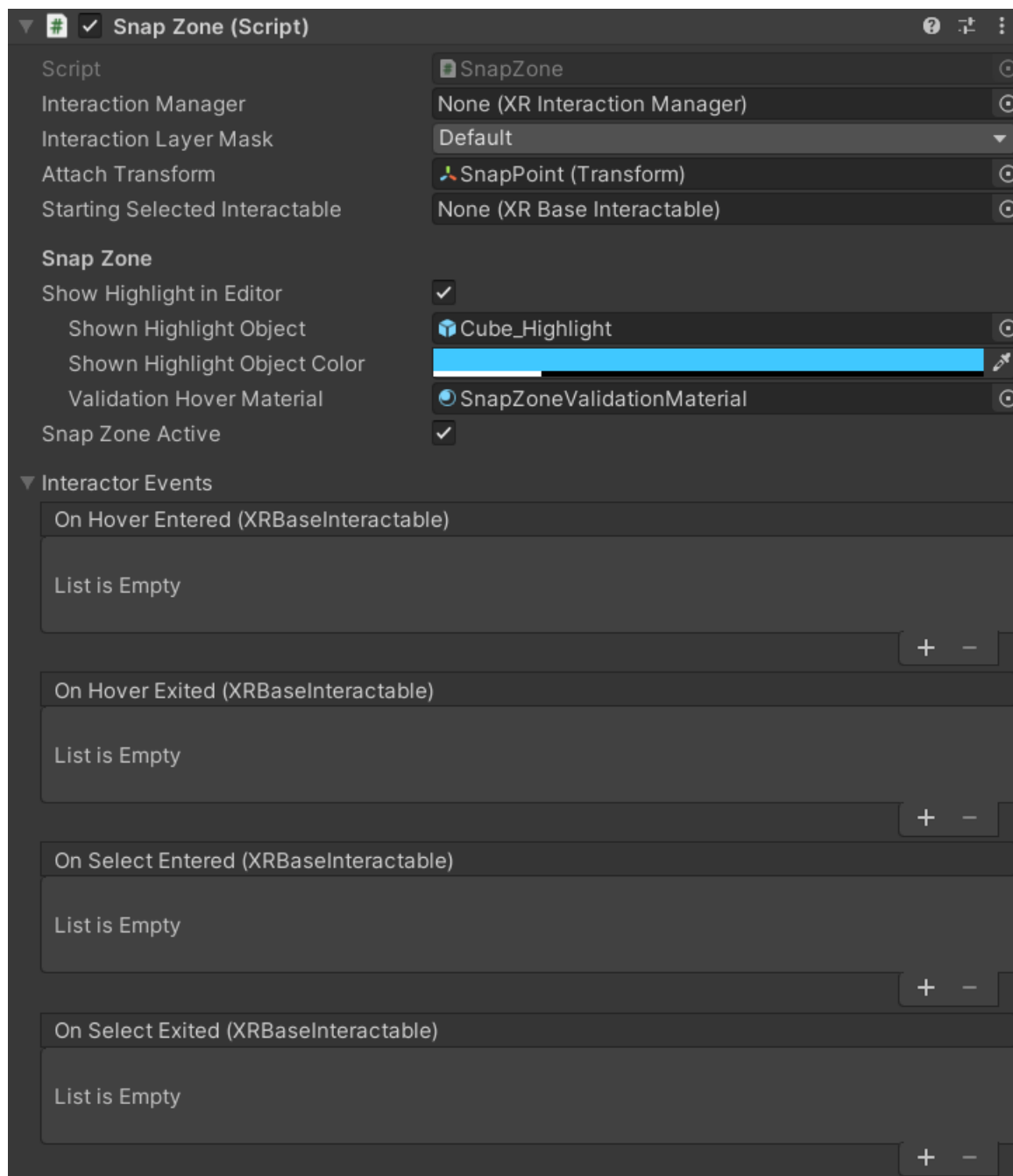
Instead of the automatic generation as described above, you can do those steps also manually. Please refer to available documentation on the `XRSocketInteractor` from Unity or related sources. You can also make changes to the automatically created snap zone to adapt it to your needs. Please note that these changes might impact the process logic.

- **Feed Forward for Snap Zones**

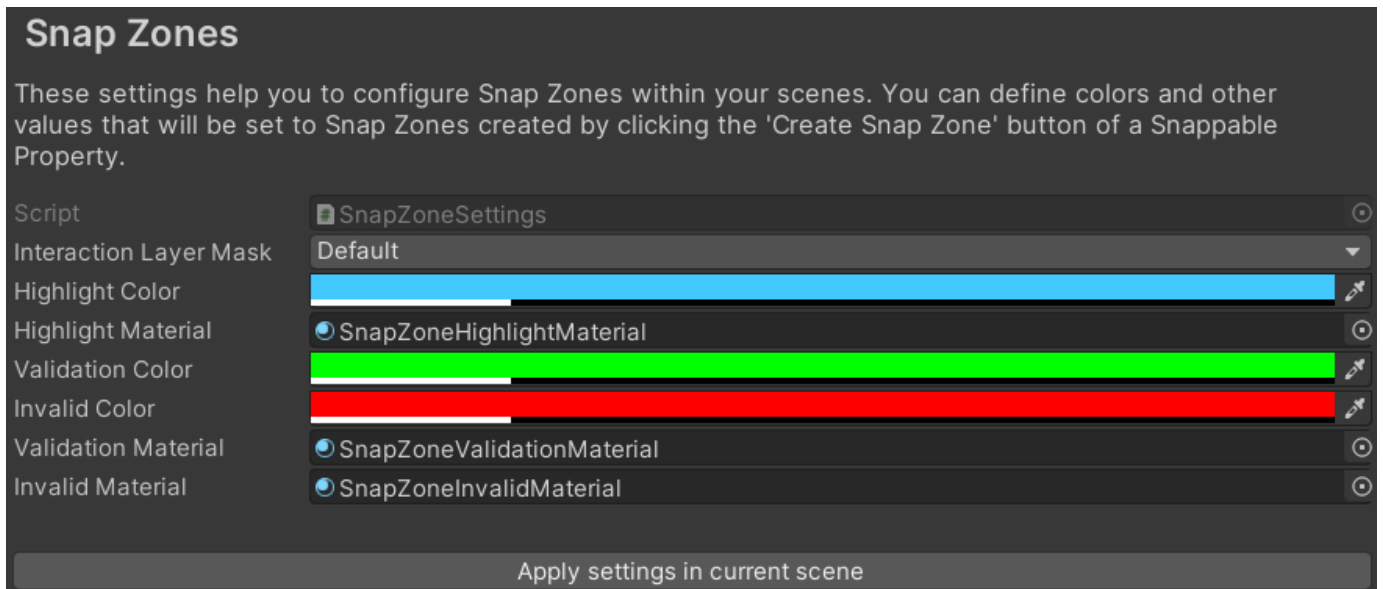
Snap zones are restricted to which objects can be snapped. This means every object can be valid (i.e. it can be snapped to this zone) or invalid (it can not be snapped to this zone) for a snap zone. This is achieved with validation components on the snap zone, for example the `Has Guid Validation` component. You can use this component to configure which specific objects or groups are accepted by the snap zone. In case you are moving a valid object into a zone, the snap zone color changes to 'Validation Color' (green), providing the user in VR with positive feedback. In case you are moving an invalid object into a zone, the snap zone color changes to 'Invalid Color' (red), giving the user the feedback that this is the wrong object for this zone. You can modify the colors and materials to be used in the Snap Zones parameters and settings.

- **Snap Zone Parameters and Settings**

To change the highlight color or validation hover material of a dedicated snap zone, navigate to the snap zone object in the Unity Inspector. You will find the Snap Zone Parameters and Settings in the script `Snap Zone`.



To change the colors and materials of all snap zones in the scene, select them in the VR Builder snap zone settings and press 'Apply settings in current scene'.



The snap zone settings can be found in the project settings in tab `VR Builder > Settings > Snap Zones`.

Configuration

- **Objects**

The `Process Scene Objects` to place (snap). The objects needs to have the `Snappable Property` and a collider component configured.

- **Zone to snap into**

This field contains the `Process Scene Object` where any of the `Objects` are required to be snapped. Make sure the object has a collider component with the `Is Trigger` property *enabled*.

- **Lock settings**

If the `Inherit Scene Object Lock State` is checked on the `Snappable Property`, the `Process Scene Object` will inherit the lock state of its parent `Process Scene Object`.

If the `Is Always Unlocked` is checked on the `Snappable Property`, the `Process Scene Object` will not be locked by the VR Builder process at runtime. It is still possible to lock the `Snappable Property` manually.

Touch Object

Description

The Touch Object condition is fulfilled when any of the `Objects` is touched by the user's controller. If a user is already touching the any of the objects while this condition is active, this condition is fulfilled immediately.

Configuration

- **Object**

The `Process Scene Objects` to be touched. The objects need to have the `Touchable Property` and a collider component configured.

- **Lock settings**

If the `Inherit Scene Object Lock State` is checked on the `Touchable Property`, the `Process Scene Object` will inherit the lock state of its parent `Process Scene Object`.

If the `Is Always Unlocked` is checked on the `Touchable Property`, the `Process Scene Object` will not be locked by the VR Builder process at runtime. It is still possible to lock the `Touchable Property` manually.

Poke Object

Description

The Poke Object condition is fulfilled when any of the `Objects` is poked by the user's controller to the specified poke depth threshold and held for the required duration in seconds. If `Must Poke All Objects` is enabled, all listed objects must be poked to fulfill the condition. If a user is already poking any of the objects while this condition is active, this condition is fulfilled immediately once the depth and duration requirements are met.

Configuration

- **Objects**

The `Process Scene Objects` to be poked. The objects need to have the `Pokable Property` and a collider component configured.

- **Must Poke All Objects**

If enabled, all listed objects must be poked to fulfill the condition. If disabled, poking any one of the objects is sufficient.

- **Poke Depth Threshold**

A value between 0 and 1 that defines how deep the poke must reach for the condition to register. A value of 0 means any contact is sufficient, while a value of 1 requires the poke to reach full depth.

- **Required Hold Duration**

The time in seconds the user must hold the poke at or beyond the depth threshold for the condition to be fulfilled. A value of 0 means the condition is fulfilled as soon as the threshold is reached.

- **Lock settings**

If the `Inherit Scene Object Lock State` is checked on the `Pokable Property`, the `Process Scene Object` will inherit the lock state of its parent `Process Scene Object`.

If the `Is Always Unlocked` is checked on the `Pokable Property`, the `Process Scene Object` will not be locked by the VR Builder process at runtime. It is still possible to lock the `Pokable Property` manually.

Remarks

- The `Pokable Property` requires an `XRSimpleInteractable` and an `XR Poke Filter` component on the object. These are added automatically when the `Pokable Property` is configured.
- The `Pokable Property` is not compatible with the `Grabbable Property`, `Usable Property`, or `Touchable Property`. Only one of these interaction properties should be used on a given object.

Poke Follow Affordance

By default, the Poke Object condition works on any `GameObject` with a `Pokable Property` and does not require the object to visually move when poked.

If you want the object to **visually move or press** in response to the poke (for example, a button that presses in when poked), you need to add the `XR Poke Follow Affordance` component. This requires a different setup from the standard poke configuration. Refer to the `pokeButton` prefab included in the project for a working example of how to set this up correctly.

This setup is specifically for cases where the object should move on poke. For all other poke interactions, the standard `Pokable Property` configuration is sufficient.

Use Object

Description

The Use Objects condition is fulfilled when any of the `Objects` are used by pressing the *Use* button of the controller while being touched or grabbed.

Configuration

- **Objects**

The `Process Scene Objects` of which one is required to be used. The `Objects` need to have the `Usable Property` and a collider component configured.

- **Lock settings**

If the `Inherit Scene Object Lock State` is checked on the `Usable Property`, the `Process Scene Object` will inherit the lock state of its parent `Process Scene Object`.

If the `Is Always Unlocked` is checked on the `Usable Property`, the `Process Scene Object` will not be locked by the VR Builder process at runtime. It is still possible to lock the `Usable Property` manually.

Teleport

Description

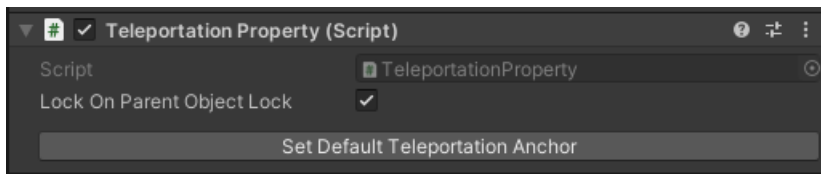
The Teleport condition is fulfilled when the user teleports to any of the referenced `Teleportation Points`. Previous teleportation actions made into a `Teleportation Point` are not considered.

If an anchor used as `Teleportation Point` has proximity detection enabled, the condition will be fulfilled not only if the user teleports to it, but also if they move close to it with continuous movement or by walking in the room. You can activate proximity detection when you need the user to be in a specific location, regardless of whether they teleport or arrive there by other locomotion types.

The provided `Teleportation Property` is based on the Unity XR Interaction Toolkit's `Teleportation Anchor`. For further reference, please check out the XR Interaction Toolkit [documentation](#).

- **Configuring a Teleportation Point**

The `Teleportation Property` can be set as a **Default Teleportation Anchor** by clicking on the `Set Default Teleportation Anchor` button. You can find it when selecting the `Teleportation Point` and viewing it in the Unity Inspector.



This will configure the attached `Teleportation Anchor`. It will provide a visual element in the Unity Editor that helps placing the `Teleportation Point` in the scene. This visual element will also be shown in the virtual world during training execution to guide the user.

Configuration

- **Teleportation Points**

The `Teleportation Properties` of which one, the user should teleport to.

Timeout

Description

The Timeout condition is fulfilled when the time specified in `Wait (in seconds)` has elapsed. This can make sense as a "fallback" condition. For example, if the user does not complete condition X in the allotted amount of time, the timeout condition will trigger leading to a different step with different consequences.

Configuration

- **Wait (in seconds)**

Set the time in seconds that should elapse before this condition is fulfilled.

Online Tutorials

We offer a constantly expanding list of [guides and tutorials](#) on our website. We encourage you to check them out to improve your VR Builder skills.

If this is your first time with VR Builder, you should start from the [Process Editor](#) and [Step Inspector](#) tutorials, which explain the basics of working with VR Builder.

In addition, you might also want to check out the guides on how to build standalone VR Builder apps on the [Oculus Quest](#) or [Pico Neo 3](#).

You can also check out some guides on the more advanced interactions, like the [series on snap zones](#).

Lastly, there are some [step-by-step tutorials](#) explaining how to work with our latest paid add-ons. Even if you don't intend to buy the relevant content, they can provide a good overview on how to build a functional process with VR Builder from scratch.

Acknowledgements

VR Builder is based on the open source edition of the [Innoactive Creator](#). While Innoactive helps enterprises to scale VR training, we adopted this tool to provide value for smaller content creators looking to streamline their VR development processes.

Like Innoactive, we believe in the value of open source and will continue to support this approach together with them and the open source community. This means you are welcome to contribute to the [VR Builder GitHub repositories](#).

Contact and Support

Join our official [Discord server](#) for quick support from the developer and fellow users. Suggest and vote on new ideas to influence the future of the VR Builder.

Make sure to review VR Builder on the [Unity Asset Store](#) if you like it. This will help us sustain the development of VR Builder.

If you have any issues, please contact contact@mindport.co. We'd love to get your feedback, both positive and constructive. By sharing your feedback you help us improve - thank you in advance! Let's build something extraordinary!

You can also visit our website at mindport.co.